

RL Topic Final Report

1. Methodology Introduction

在 Austria，我用了 PPO 演算法加上 gSDE(generalized State Dependent Exploration)以克服這個複雜環境中的 Exploration 問題，並用了自己設計的 Reward Function。至於 Circle，由於這個環境就為簡單，故直接使用了與 Austria 的相同實驗設計。

2. Experiment Design and Implementation

實作上我用了 stable baselines 3 提供的 PPO + gSDE，並用了 wandb callback 在記錄訓練過程與參數；在模型架構中，SB3 是先使用了 NatureCNN 擔任 state 的 feature extractor，再接到 (x, 1) 的 value head 和 (x, a) 的 actor head，其中 a 是 Action dimension。

NatureCNN:

```
self.cnn = nn.Sequential(
    nn.Conv2d(n_input_channels, out_channels=32, kernel_size=8, stride=4, padding=0),
    nn.ReLU(),
    nn.Conv2d(in_channels=32, out_channels=64, kernel_size=4, stride=2, padding=0),
    nn.ReLU(),
    nn.Conv2d(in_channels=64, out_channels=64, kernel_size=3, stride=1, padding=0),
    nn.ReLU(),
    nn.Flatten(),
)
```

後面連接一個(x, 512)的 linear layer 和 ReLU，其中 x 是 NatureCNN 的輸出。

環境預處理：(Grayscale, resize to 84x84, 8 frame stack)

Hyperparameters:

- Subprocesses = 64
- Learning rate = 3e-4
- Total timesteps = 5e7
- N_steps (update interval) = 1024
- Batch size = 64

Packages:

- stable_baselines3 2.4.0
- torch 2.5.1

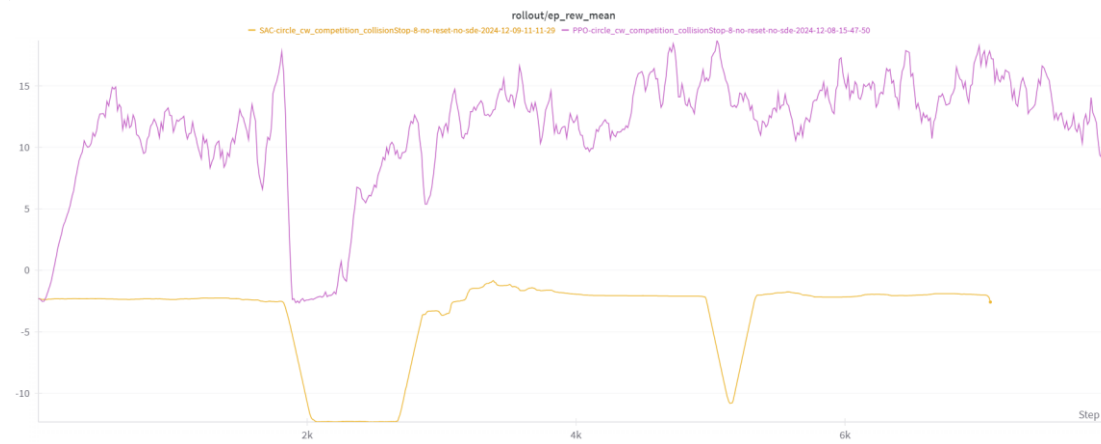
Resources (borrowed from lab):

- INTEL XEON GOLD 5515
- A6000

3. Method Comparison and Evaluation

初始實驗：SAC vs PPO

在最一開始使用沒使用 gSDE，並使用預設的 reward function 進行實驗，以決定要使用 PPO 或 SAC 進行後續實驗，看出就算在 circle 的情況下，SAC 還是無法太好學習，故使用 PPO 進行後續實驗。



第二次實驗：reward function 初步設計 (沒圖，因為 wandb 上沒記錄到)

注意到 scenario 裡面可以設計 collision_reward、progress_reward、frame_reward，故先實驗一些組合，看能不能有些想法以設計更複雜的 reward function。

先是注意到 collision，由於我們不想讓車去撞牆，故 collision 先給他大一點，例如-100；再來是 progress，從前幾次實驗可以注意到 progression reward 只給 1.0 的話，總 reward 也至多在 10 上下，所以這邊也要調大一些，例如 100 左右；至於 frame reward 先給 1.0，當作是保持存活的獎勵。

在上面的實驗，發現 agent 可能會由於害怕撞牆，故直接放棄 progress reward，直接停在原地拿 frame reward。

故將 frame reward 改成-1.0，希望能透過這個鼓勵拿到 progress reward，發現 agent 會開始嘗試前進，但變成勇於撞牆以減少 frame penalty；這邊猜測是由於不管有沒有前進都會拿到 penalty，所以要設計一個要是前進就不會有 penalty，停滯不前才會拿到 penalty 的 reward function。

第三次實驗：reward function 進階設計 (同上，沒圖)

改寫原本的 RaceEnv 中的 step，以修改 reward function。先延續上面的設計，將 progress 和 collision reward 設為 100 和 -100，另外在參考 progress，若有往前開 (progress 變大) 則給予 $100 * \text{progress}$ ，否則給予 -1.0 的 penalty，發現這樣改有助於 agent 學習，但 agent 有不經過 checkpoint 的行為傾向。

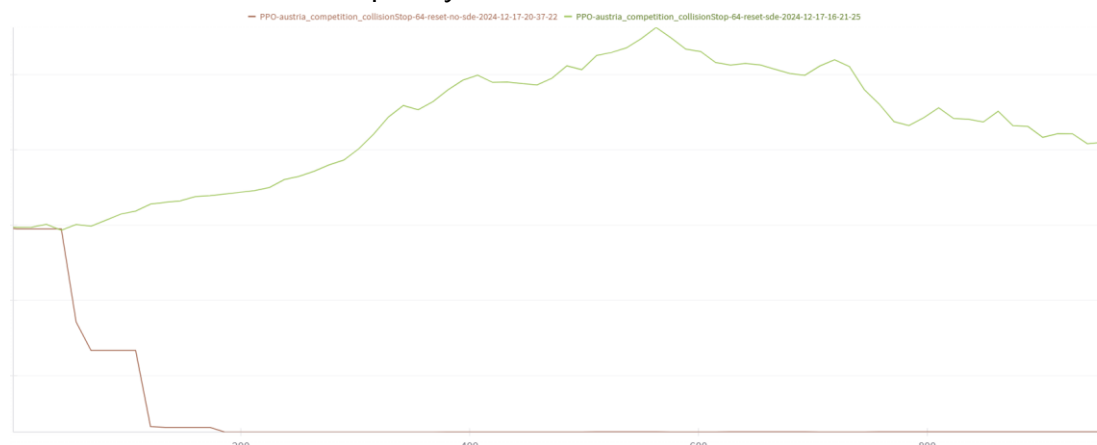
後續再加上，若 checkpoint 被更新過，則給予 10 的 reward，發現行為也有改善。

另外注意到 info 內還有 motor 和 steering 的資訊，由於我們希望 agent 盡量快行駛完一圈以減少時間 penalty，而 motor 可以加速，故再將 reward 加上 $1 * \text{motor}$ ，希望可以鼓勵飆車。

設計至此就是最後拿來訓練的 reward function 了。

第四次實驗：gSDE

發現上面訓練的 agent 曲線不太穩定，偶爾會收斂到很差的地方，故想利用 gSDE 來增進 exploration 的行為，可以看到有 gSDE 的行為就好很多，若沒有則常常會收斂到很差的 policy (如下圖棕線)。



4. Challenges and Learning Points

- 我認為最大的挑戰還是在於自己設計 reward function 並想辦法使其變好，還好在這邊的 scenario 有給予三個初始提示，讓我先觀察其中三個變數，後面再利用環境給予的 info 得出更好的 reward function。
- 再這些實驗中，學到了一個好的 reward function 不應該只是鼓勵 agent「活著」，而是適當給予 penalty 以避免 agent 死亡或

停滯不前，並在其嘗試前進時給予 reward。

5. Future Work

- a. 現在有許多 Imitation Learning 的演算法，在這次實驗中沒有試過，也沒有適當的 dataset 訓練，希望將來有機會使用 IL 來克服這個環境。
- b. 最終練出的 agent 仍然有點不穩定，可能還需要更多的訓練來讓他有更完整的 policy，也可能是 reward function 還不夠完善。